SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.Kushal	**Reg. No.:**	1924260243
Section / Batch:	2026	**Date:**	20/10/26

Code of Conduct
I M.Kushal (1924260243) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M.Kushal

Instructions
- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as "analyzing, "analysis", and "analytical". The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- **Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.**

Sol: CODE

```
import pandas as pd

class SearchEngineMorphAnalyzer:
    def __init__(self):
        self.canonical_root = "analyze"
        self.rules = {
            "analyzing": {
                "prefix": "None",
                "suffix": "-ing",
                "type": "Inflectional",
                "pos_shift": "Verb -> Verb (Participle)",
                "structure": "analyze - e + -ing"
            },
            "analysis": {
                "prefix": "None",
                "suffix": "-sis",
                "type": "Derivational",
                "pos_shift": "Verb -> Abstract Noun",
                "structure": "analyze -> analysis (-ze to -sis)"
            },
            "analytical": {
                "prefix": "None",
                "suffix": "-tic, -al",
                "type": "Derivational",
                "pos_shift": "Verb -> Qualitative Adjective",
                "structure": "analyze + -tic + -al"
            }
        }

    def process_words(self, word_list):
        report = []
        for word in word_list:
            w_lower = word.lower().strip()
            if w_lower in self.rules:
                info = self.rules[w_lower]
                report.append({
                    "Original Word": word,
                    "Extracted Root": self.canonical_root,
                    "Identified Affix(es)": info["suffix"],
                    "Transformation Type": info["type"],
                    "POS Shift": info["pos_shift"],
                    "Parsed Structure": info["structure"],
                    "Normalized Output": self.canonical_root
                })
```

```
    return pd.DataFrame(report)

# Execution
words_q1 = ["analyzing", "analysis", "analytical"]
analyzer = SearchEngineMorphAnalyzer()
df_q1 = analyzer.process_words(words_q1)

print("=== Q1: Search Engine Morphological Processing Report ===")
print(df_q1.to_string(index=False))
```

OUTPUT:

```
=== Q1: Search Engine Morphological Processing Report ===
Original Word Extracted Root Identified Affix(es) Transformation Type                  POS Shift                Parsed Structure Normalized Output
   analyzing        analyze               -ing       Inflectional     Verb -> Verb (Participle)              analyze - e + -ing          analyze
    analysis        analyze               -sis       Derivational        Verb -> Abstract Noun analyze -> analysis (-ze to -sis)          analyze
  analytical        analyze          -tic, -al       Derivational Verb -> Qualitative Adjective             analyze + -tic + -al          analyze
```

2. An AI-based sentiment analysis platform receives product reviews containing words like “disagree”, “agreement”, and “agreeable”. The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- **Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.**

Sol: CODE

```
import pandas as pd

class SentimentMorphParser:
    def __init__(self):
        self.root_base = "agree"
        self.parsing_db = {
            "disagree": {
                "prefix": "dis-",
                "suffix": "None",
                "category": "Derivational (Prefixation)",
                "semantics": "Polarity Inversion (Positive -> Negative)"
            },
            "agreement": {
                "prefix": "None",
                "suffix": "-ment",
                "category": "Derivational (Suffixation)",
                "semantics": "Nominalization (State/Instance Noun)"
            },
            "agreeable": {
                "prefix": "None",
                "suffix": "-able",
                "category": "Derivational (Suffixation)",
                "semantics": "Adjectival Property (Inclined to consent)"
            }
        }

    def parse_sentiment_words(self, words):
        results = []
        for word in words:
            w_clean = word.lower().strip()
            if w_clean in self.parsing_db:
                entry = self.parsing_db[w_clean]
```

```
            results.append({
                "Original Word": word,
                "Identified Prefix": entry["prefix"],
                "Root Word": self.root_base,
                "Identified Suffix": entry["suffix"],
                "Transformation Category": entry["category"],
                "Semantic Interpretation": entry["semantics"],
                "Normalized Base Form": self.root_base
            })
        return pd.DataFrame(results)

# Execution
words_q2 = ["disagree", "agreement", "agreeable"]
parser = SentimentMorphParser()
df_q2 = parser.parse_sentiment_words(words_q2)

print("=== Q2: Sentiment Morphological Parser Report ===")
print(df_q2.to_string(index=False))
```

OUTPUT:

```
=== Q2: Sentiment Morphological Parser Report ===
Original Word Identified Prefix Root Word Identified Suffix    Transformation Category                Semantic Interpretation Normalized Base Form
     disagree              dis-     agree              None Derivational (Prefixation) Polarity Inversion (Positive -> Negative)               agree
    agreement              None     agree             -ment Derivational (Suffixation)      Nominalization (State/Instance Noun)               agree
    agreeable              None     agree             -able Derivational (Suffixation) Adjectival Property (Inclined to consent)               agree
```

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as "govern", "government", and "governance". The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- **Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.**

Sol: CODE

```
import pandas as pd

class TopicModelingNormalizer:
    def __init__(self):
        self.base_root = "govern"
        self.hierarchy_map = {
            "govern": {
                "affix": "None",
                "level": "Level 0 (Base)",
                "hierarchy": "[Root: govern]",
                "feature_key": "govern"
            },
            "government": {
                "affix": "-ment",
                "level": "Level 1 (Derivation)",
                "hierarchy": "[govern] -> [+ment] (Institutional Noun)",
                "feature_key": "govern"
            },
            "governance": {
                "affix": "-ance",
                "level": "Level 1 (Derivation)",
                "hierarchy": "[govern] -> [+ance] (Process Noun)",
                "feature_key": "govern"
            }
        }
```

```
    def normalize_corpus_tokens(self, tokens):
        output = []
        for token in tokens:
            t_clean = token.lower().strip()
            if t_clean in self.hierarchy_map:
                meta = self.hierarchy_map[t_clean]
                output.append({
                    "Original Word": token,
                    "Root Form": self.base_root,
                    "Detected Affix(es)": meta["affix"],
                    "Derivational Level": meta["level"],
                    "Derivational Hierarchy": meta["hierarchy"],
                    "Normalized Representation": self.base_root,
                    "Topic Model Feature Key": meta["feature_key"]
                })
        return pd.DataFrame(output)

# Execution
words_q3 = ["govern", "government", "governance"]
normalizer = TopicModelingNormalizer()
df_q3 = normalizer.normalize_corpus_tokens(words_q3)

print("=== Q3: News Normalization & Topic Modeling Output ===")
print(df_q3.to_string(index=False))
```

OUTPUT:

```
=== Q3: News Normalization & Topic Modeling Output ===
Original Word Root Form Detected Affix(es)   Derivational Level                 Derivational Hierarchy Normalized Representation Topic Model Feature Key
        govern    govern                None      Level 0 (Base)                      [Root: govern]                    govern                  govern
    government    govern             -ment Level 1 (Derivation) [govern] -> [+ment] (Institutional Noun)                    govern                  govern
    governance    govern             -ance Level 1 (Derivation)       [govern] -> [+ance] (Process Noun)                    govern                  govern
```

4. A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.
Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers.
Analyze how prefix addition and suffix transformation affect meaning and word class.
Apply the process to decompose each word and map them to a unified base representation.
Derive the morphological structure and normalized root for each input word.

- **Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.**

Sol: CODE

```
import pandas as pd

class DocumentClassifierParser:
    def __init__(self):
        self.canonical_base = "activate"
        self.derivational_db = {
            "activate": {
                "prefix": "None",
                "suffix": "-ate",
                "sequence": "active (Adj) + -ate",
                "pos_shift": "Adjective -> Causative Verb",
                "effect": "Causative action ('make active')",
                "parsed": "[activate]"
            },
```

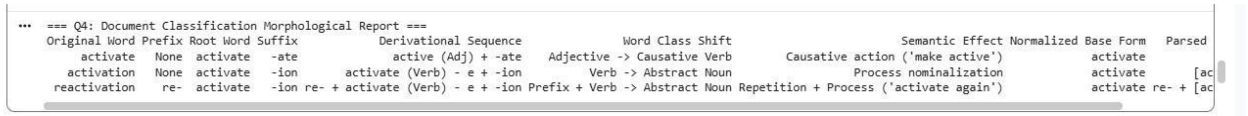
```
            "activation": {
                "prefix": "None",
                "suffix": "-ion",
                "sequence": "activate (Verb) - e + -ion",
                "pos_shift": "Verb -> Abstract Noun",
                "effect": "Process nominalization",
                "parsed": "[activate] + -ion"
            },
            "reactivation": {
                "prefix": "re-",
                "suffix": "-ion",
                "sequence": "re- + activate (Verb) - e + -ion",
                "pos_shift": "Prefix + Verb -> Abstract Noun",
                "effect": "Repetition + Process ('activate again')",
                "parsed": "re- + [activate] + -ion"
            }
        }

    def parse_document_terms(self, terms):
        report_data = []
        for term in terms:
            t_clean = term.lower().strip()
            if t_clean in self.derivational_db:
                data = self.derivational_db[t_clean]
                report_data.append({
                    "Original Word": term,
                    "Prefix": data["prefix"],
                    "Root Word": self.canonical_base,
                    "Suffix": data["suffix"],
                    "Derivational Sequence": data["sequence"],
                    "Word Class Shift": data["pos_shift"],
                    "Semantic Effect": data["effect"],
                    "Normalized Base Form": self.canonical_base,
                    "Parsed Representation": data["parsed"]
                })
        return pd.DataFrame(report_data)

# Execution
words_q4 = ["activate", "activation", "reactivation"]
classifier_parser = DocumentClassifierParser()
df_q4 = classifier_parser.parse_document_terms(words_q4)

print("=== Q4: Document Classification Morphological Report ===")
print(df_q4.to_string(index=False))
```

OUTPUT:



```
=== Q4: Document Classification Morphological Report ===
Original Word Prefix Root Word Suffix            Derivational Sequence              Word Class Shift                      Semantic Effect Normalized Base Form   Parsed
     activate   None  activate   -ate              active (Adj) + -ate    Adjective -> Causative Verb       Causative action ('make active')             activate
   activation   None  activate   -ion       activate (Verb) - e + -ion          Verb -> Abstract Noun                Process nominalization             activate       [ac
 reactivation    re-  activate   -ion re- + activate (Verb) - e + -ion Prefix + Verb -> Abstract Noun Repetition + Process ('activate again')            activate re- + [ac
```

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.
Identify suffix patterns and map all variations to a common base form without altering meaning.
Apply the process to extract root forms and classify each transformation.
Determine the morphological breakdown and normalized output for each word.

- **Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.**

Sol: CODE

```
import pandas as pd

class InflectionalNormalizer:
    def __init__(self):
        self.base_verb = "create"
        self.inflection_rules = {
            "s": {
                "category": "3rd Person Singular Present",
                "rule": "Strip '-s'",
                "breakdown": "create + -s"
            },
            "ing": {
                "category": "Present Participle / Progressive",
                "rule": "Strip '-ing' and restore 'e'",
                "breakdown": "create - e + -ing"
            }
        }

    def normalize_word(self, word):
        w_lower = word.lower().strip()

        # Exact base match
        if w_lower == self.base_verb:
            return {
                "Original Word": word,
                "Identified Suffix": "None",
                "Grammatical Category": "Base Form (Infinitive)",
                "Normalization Rule": "Identity (No change)",
                "Extracted Root": self.base_verb,
                "Morphological Breakdown": "create + Ø",
                "Normalized Base Form": self.base_verb,
                "Search Index Key": self.base_verb
            }

        # Suffix matching
        for sfx, info in self.inflection_rules.items():
            if w_lower.endswith(sfx):
                return {
                    "Original Word": word,
                    "Identified Suffix": f"-{sfx}",
                    "Grammatical Category": info["category"],
                    "Normalization Rule": info["rule"],
                    "Extracted Root": self.base_verb,
```

```
                "Morphological Breakdown": info["breakdown"],
                "Normalized Base Form": self.base_verb,
                "Search Index Key": self.base_verb
            }

        return {
            "Original Word": word,
            "Identified Suffix": "Unknown",
            "Grammatical Category": "Unknown",
            "Normalization Rule": "None",
            "Extracted Root": word,
            "Morphological Breakdown": word,
            "Normalized Base Form": word,
            "Search Index Key": word
        }

    def process_batch(self, word_list):
        records = [self.normalize_word(w) for w in word_list]
        return pd.DataFrame(records)

# Execution
words_q5 = ["create", "creates", "creating"]
normalizer_q5 = InflectionalNormalizer()
df_q5 = normalizer_q5.process_batch(words_q5)

print("=== Q5: Search Optimization Inflectional Normalization Output ===")
print(df_q5.to_string(index=False))
```

OUTPUT:

```
=== Q5: Search Optimization Inflectional Normalization Output ===
Original Word Identified Suffix            Grammatical Category          Normalization Rule Extracted Root Morphological Breakdown Normalized Base Form Search Index Key
       create              None          Base Form (Infinitive)        Identity (No change)         create              create + ∅               create           create
      creates                -s      3rd Person Singular Present                Strip '-s'         create             create + -s               create           create
     creating              -ing Present Participle / Progressive Strip '-ing' and restore 'e'         create       create - e + -ing               create           create
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____